

# std::execution — Sender Algorithm Cheat Sheet (C++26)

Verified August 2026 against the C++26 draft clauses ([exec.\*]), [NVIDIA/stdexec](#) (main), and [bemanproject/execution](#) (main).

**Namespaces:** standard entities live in `std::execution` (except `sync_wait`, which is in `std::this_thread`). `stdexec` puts standard-track entities in `stdexec::` and extensions in `exec::`. Beman uses `beman::execution::`.

**Column key:** ✓ implemented · (✓) implemented with caveat in footnote · — not available.

---

## Sender factories — start a pipeline

| Algorithm                    | What it does                                                                                    | stdexec | Beman |
|------------------------------|-------------------------------------------------------------------------------------------------|---------|-------|
| <code>just(vs...)</code>     | Completes immediately with the given values                                                     | ✓       | ✓     |
| <code>just_error(e)</code>   | Completes immediately on the error channel                                                      | ✓       | ✓     |
| <code>just_stopped()</code>  | Completes immediately on the stopped channel                                                    | ✓       | ✓     |
| <code>schedule(sch)</code>   | Completes (with nothing) on the scheduler's execution context — the root of most pipelines      | ✓       | ✓     |
| <code>read_env(query)</code> | Completes with a value read from the receiver's environment (e.g. <code>get_stop_token</code> ) | ✓       | ✓     |

## Sender adaptors — transform and compose

| Algorithm                             | What it does                                                                                          | stdexec | Beman |
|---------------------------------------|-------------------------------------------------------------------------------------------------------|---------|-------|
| <code>then(sndr, f)</code>            | Transform the value(s) with an ordinary function                                                      | ✓       | ✓     |
| <code>upon_error(sndr, f)</code>      | Transform an error into a value (recovery)                                                            | ✓       | ✓     |
| <code>upon_stopped(sndr, f)</code>    | Turn cancellation into a value                                                                        | ✓       | ✓     |
| <code>let_value(sndr, f)</code>       | <code>f</code> returns a <b>sender</b> ; chains a dependent async step                                | ✓       | ✓     |
| <code>let_error(sndr, f)</code>       | Async recovery: error → new sender                                                                    | ✓       | ✓     |
| <code>let_stopped(sndr, f)</code>     | Async fallback on cancellation                                                                        | ✓       | ✓     |
| <code>starts_on(sch, sndr)</code>     | Run the whole sender on <code>sch</code>                                                              | ✓       | ✓     |
| <code>continues_on(sndr, sch)</code>  | Resume everything downstream on <code>sch</code>                                                      | ✓       | ✓     |
| <code>on(sch, sndr)</code>            | Round trip: run <code>sndr</code> on <code>sch</code> , then return to the original scheduler (P3175) | ✓       | ✓     |
| <code>schedule_from(sch, sndr)</code> | Customization point underlying <code>continues_on</code> ; rarely called directly                     | ✓       | ✓     |
| <code>when_all(sndrs...)</code>       | Run concurrently; join into one completion.                                                           | ✓       | ✓     |

| Algorithm                                  | What it does                                                                                         | stdexec          | Beman |
|--------------------------------------------|------------------------------------------------------------------------------------------------------|------------------|-------|
|                                            | First error/stop cancels the rest                                                                    |                  |       |
| <code>when_all_with_variant(...)</code>    | <code>when_all</code> over senders with multiple value signatures                                    | ✓                | ✓     |
| <code>into_variant(sndr)</code>            | Collapse multiple value signatures into one<br><code>variant&lt;tuple&lt;...&gt;...&gt;</code> value | ✓                | ✓     |
| <code>stopped_as_optional(sndr)</code>     | Stopped → value channel as empty <code>optional</code>                                               | ✓                | ✓     |
| <code>stopped_as_error(sndr, e)</code>     | Stopped → error channel with your error                                                              | ✓                | ✓     |
| <code>split(sndr)</code>                   | Single-shot → multi-shot: many consumers, work runs once                                             | (✓) <sup>1</sup> | ✓     |
| <code>bulk(sndr, pol, shape, f)</code>     | Parallel loop over <code>[0, shape)</code> under an execution policy                                 | (✓) <sup>2</sup> | ✓     |
| <code>bulk_chunked / bulk_unchunked</code> | <code>bulk</code> variants with explicit chunking control (P3481)                                    | —                | ✓     |
| <code>write_env(sndr, env)</code>          | Run child with additional environment entries                                                        | ✓                | ✓     |
| <code>unstoppable(sndr)</code>             | Hide stop requests from the child — a cancellation firewall                                          | ✓                | ✓     |

| Algorithm                           | What it does                                                                            | stdexec | Beman |
|-------------------------------------|-----------------------------------------------------------------------------------------|---------|-------|
| <code>stop_when(sndr, token)</code> | Also request stop when an external token fires                                          | ✓       | ✓     |
| <code>affine</code>                 | Scheduler-affinity adaptor used by <code>task</code> to return to its scheduler (P3552) | ✓       | ✓     |

## Sender consumers — leave sender-land

| Algorithm                                              | What it does                                                                                                                              | stdexec | Beman |
|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|---------|-------|
| <code>this_thread::sync_wait(sndr)</code>              | Block until done; <code>optional&lt;tuple&lt;Vs...&gt;&gt;</code> (empty = stopped, errors rethrow). Requires exactly one value signature | ✓       | ✓     |
| <code>this_thread::sync_wait_with_variant(sndr)</code> | Same, for senders with multiple value signatures                                                                                          | ✓       | ✓     |

## Structured concurrency scopes (P3149)

| Facility                           | What it does                                                      | stdexec | Beman |
|------------------------------------|-------------------------------------------------------------------|---------|-------|
| <code>counting_scope</code>        | Tracks spawned work; join it before the scope dies                | ✓       | ✓     |
| <code>simple_counting_scope</code> | Same, minus stop-request support                                  | ✓       | ✓     |
| <code>spawn(sndr, tok)</code>      | Fire-and-forget into a scope (this is where allocation is opt-in) | ✓       | ✓     |

| Facility                             | What it does                                           | stdexec | Beman |
|--------------------------------------|--------------------------------------------------------|---------|-------|
| <code>spawn_future(sndr, tok)</code> | Spawn eagerly, get a sender for the result             | ✓       | ✓     |
| <code>associate(sndr, tok)</code>    | Tie a sender's lifetime to a scope without starting it | ✓       | ✓     |

## Coroutine integration (P2300 + P3552)

| Facility                                     | What it does                                                       | stdexec | Beman |
|----------------------------------------------|--------------------------------------------------------------------|---------|-------|
| <code>task&lt;T, Env&gt;</code>              | Standard coroutine task type; <code>co_await</code> senders inside | ✓       | ✓     |
| <code>task_scheduler</code>                  | Type-erased scheduler for <code>task</code> environments           | ✓       | ✓     |
| <code>as_awaitable(sndr)</code>              | Make any sender awaitable in a coroutine                           | ✓       | ✓     |
| <code>with_awaitable_senders&lt;P&gt;</code> | Promise-base that lets a coroutine await senders                   | ✓       | ✓     |

## Schedulers & contexts in the standard

| Facility                                                   | What it does                                                               | stdexec | Beman |
|------------------------------------------------------------|----------------------------------------------------------------------------|---------|-------|
| <code>run_loop</code>                                      | Single-threaded manually-driven context (what <code>sync_wait</code> uses) | ✓       | ✓     |
| <code>parallel_scheduler / get_parallel_scheduler()</code> | The standard system-provided parallel context (P2079)                      | ✓       | ✓     |

---

## Renamed or removed on the way to C++26

| Old (pre-R10 / early drafts)          | C++26                                  | Notes                                                                                                       |
|---------------------------------------|----------------------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>transfer</code>                 | <code>continues_on</code>              | stdexec's older releases spell it <code>transfer</code>                                                     |
| <code>on</code> (old meaning)         | <code>starts_on</code>                 | <code>on</code> now names the P3175 round-trip adaptor                                                      |
| <code>read</code>                     | <code>read_env</code>                  |                                                                                                             |
| <code>tag_invoke</code> customization | member functions                       | changed in P2300R10, the adopted revision                                                                   |
| <code>ensure_started</code>           | <i>removed</i>                         | use <code>counting_scope</code> + <code>spawn_future</code> ; survives as <code>exec::ensure_started</code> |
| <code>start_detached</code>           | <i>removed</i>                         | use <code>spawn</code> ; survives as <code>exec::start_detached</code>                                      |
| <code>execute</code>                  | <i>removed</i>                         |                                                                                                             |
| <code>bulk(sndr, shape, f)</code>     | <code>bulk(sndr, pol, shape, f)</code> | execution policy added by P3481                                                                             |

## Notable stdexec-only extensions (`exec::`, no standard equivalent)

| Facility                                                 | What it does                                                                      |
|----------------------------------------------------------|-----------------------------------------------------------------------------------|
| <code>async_scope</code>                                 | Pre-standard ancestor of <code>counting_scope</code> (the deck's examples use it) |
| <code>when_any(sndrs...)</code>                          | First completion wins; the rest are cancelled                                     |
| <code>repeat_effect_until</code> / <code>repeat_n</code> | Looping combinators                                                               |
| <code>finally(sndr, cleanup)</code>                      | Unconditional async cleanup (async RAII) <sup>3</sup>                             |

| Facility                                                                                                           | What it does                                                                  |
|--------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <code>any_sender_of&lt;Sigs...&gt;</code>                                                                          | Type-erased sender (this is an allocation point)                              |
| <code>into_tuple(sndr)</code>                                                                                      | Pack all values into one tuple value                                          |
| <code>create(fn)</code>                                                                                            | Build a sender from a callback-style initiation function — handy for wrapping |
| <code>static_thread_pool</code>                                                                                    | Concrete fixed-size thread-pool scheduler                                     |
| <code>io_uring_context</code>                                                                                      | Linux io_uring run context — timers/scheduling only, no read/write ops        |
| <code>libdispatch_queue</code>                                                                                     | Apple Grand Central Dispatch scheduler                                        |
| <code>single_thread_context</code> ,<br><code>timed_thread_scheduler</code> ,<br><code>trampoline_scheduler</code> | Assorted single-thread / timed / stack-safe contexts                          |

---

## Footnotes

1. `stdexec` currently ships `split` as an `exec::` extension rather than in `stdexec::`; Beman implements the standard `[exec.split]` version.
2. `stdexec`'s `bulk` still has the pre-P3481 shape (no policy parameter, no chunked variants).
3. `finally` appears in `stdexec`'s standard-track namespace but is **not** in C++26.

**Caveats.** Beman self-reports "under development, not ready for production"; its status doc shows a few items (`bulk`, `sync_wait`, `task`, `scopes`, `spawn`) as implemented and tested but with review/documentation still catching up. `stdexec` remains experimental with ongoing API churn. Verify against each repo before relying on a specific spelling — this sheet reflects August 2026.